

1. What is Pyspark?

PySpark is an Apache Spark interface in Python. It is used for collaborating with Spark using APIs written in Python. It also supports Spark's features like Spark DataFrame, Spark SQL, Spark Streaming, Spark MLlib and Spark Core. It provides an interactive PySpark shell to analyze structured and semi-structured data in a distributed environment. PySpark supports reading data from multiple sources and different formats. It also facilitates the use of RDDs (Resilient Distributed Datasets). PySpark features are implemented in the py4j library in python.

2. What are the characteristics of PySpark?

Abstracted Nodes: This means that the individual worker nodes cannot be addressed.

Spark API: PySpark provides APIs for utilizing Spark features.

Map-Reduce Model: PySpark is based on Hadoop's Map-Reduce model this means that the programmer provides the map and the reduced functions.

Abstracted Network: Networks are abstracted in PySpark which means that the only possible communication is implicit communication.

3) What are RDDs in PySpark? RDDs expand to Resilient Distributed Datasets. These are the elements that are used for running and operating on multiple nodes to perform parallel processing on a cluster. Since RDDs are suited for parallel processing, they are immutable elements. This means that once we create RDD, we cannot modify it. RDDs are also fault-tolerant which means that whenever failure happens, they can be recovered automatically. Multiple operations can be performed on RDDs to perform a certain task. Transformation: These operations when applied on RDDs result in the creation of a new RDD.

```
from pyspark import SparkContext
```

```
sc = SparkContext("local", "Transformation Demo") words_list = sc.parallelize (
["pyspark", "interview", "questions", "at", "interviewbit"]) filtered_words =
words_list.filter(lambda x: 'interview' in x) filtered = filtered_words.collect()
print(filtered) Action: These operations instruct Spark to perform some computations
on the RDD and return the result to the driver. It sends data from the Executer to the
driver. count(), collect(), take() are some of the examples.
```

```
from pyspark import SparkContext sc = SparkContext("local", "Action Demo") words =
sc.parallelize ( ["pyspark", "interview", "questions", "at", "interviewbit"]) counts =
words.count() print("Count of elements in RDD -> ", counts)
```

